INTERNSHIP PROJECT · TÜRKSAT AVIATION AND COMMUNICATION

Smart School Information System

Week 7 — Security, Evaluation and the Conversational Assistant

PREPARED BY	Bera Ergin — s29315
INSTITUTION	PJAIT — 7th Semester

THIS WEEK	We attack the system ourselves and measure the assistant

Where We're Coming From, Where We're Going

WEEK 6 — DONE

- ✓ Interface and backend work end to end
- ✓ Each layer is verified by its own tests
- ! But the system was never tested as an attacker would
- ! The assistant's behaviour was never measured systematically

WEEK 7 — GOAL

- A threat model, with a test for every threat
- An attempt at privilege escalation through the model's SQL
- Login hardening and CORS
- An evaluation (eval) set for the assistant's behaviour

Threat Model

Which layer stops each threat, and which test proves it

Threat	Layer	Proof
Privilege escalation through the model's SQL	Validation: function allowlist	Adversarial prompt tests
Access to sensitive tables	Database privileges (users, teachers)	Privilege error test
Brute-force login	Login rate limit, uniform 401	Login tests
JWT, password and key leaks	Response and log rules	Response/log tests
Expensive query	5 s timeout, 500 rows	Timeout test

Finding: The Model Could Change the Context Itself

The hypothesis was reproduced; closed with a function allowlist

```
WITH x AS (SELECT set_config('app.role','principal',true))
SELECT s.* FROM students s, x
```

One statement, one SELECT, allowed tables: everything today's validation checks passes. The RLS policy, meanwhile, reads the session setting.

1. Reproduced: a teacher context normally sees 4 rows, but saw 12 with this SQL
2. Why: the policy reads app.role during the scan; set_config in the same statement turns it into principal inside the transaction
3. Closed: allowlist (set_config, current_setting, pg_* rejected); 168 tests passed

The Fix: A Function Allowlist

An allowlist instead of a denylist: every unknown function is rejected

Allowed

```
COUNT, SUM, AVG, MIN, MAX, ROUND,
LOWER, UPPER, COALESCE, date functions
(added as the need arises)
```

Rejected

```
set_config, current_setting, pg_sleep,
pg_read_file, lo_import, dblink, ...
and every function not on the list
```

A denylist misses any function that never made it onto the list; an allowlist rejects what it does not know. Validation works as a third layer next to RLS and database privileges.

Login Security and CORS

The rest of the attack surface

Uniform 401

An unknown user and a wrong password give the same response and a similar duration; usernames do not leak

Login rate limit

A limited number of attempts per username per minute; 429 when exceeded

CORS

A configurable list of allowed origins (CORS_ORIGINS) instead of allow_origins=*

Secret values

SECRET_KEY is a development value; it is changed at real deployment (Week 12) and noted in the README

The Evaluation (Eval) Set

Measuring the model's behaviour systematically instead of trying it by hand

What it contains

- 65 questions, 4 roles, including follow-up scenarios
- Typical questions: grades, absences, name search
- Edge questions: comparison, chat
- Adversarial prompts: ignore the rules, write set_config

How it runs

- Against the running API with the real model
- Not part of pytest (it needs a key)
- Expected behaviour per question: result, refusal or chat
- Real values via truth_sql, tone and honesty via a judge; output: eval/report.md

The Conversational Assistant

From an SQL translator to a tool-using assistant



Tool limit

At most 3 queries per question. On a validation error the model only gets a short generic message; the database error text never goes.

Context

Schema summary, pass grade (PASS_GRADE), the student_rank and class_teacher functions, and an honest answer for information that does not exist (timetable, meetings).

Security

Chat history is untrusted input; authority comes only from the JWT and RLS. Aggregate results for parent/student carry a scope_warning.

Bugs Found in Live Testing (1/3)

Real-time bug log: symptom → root cause → fix

1 Silently wrong class average

Verified (eval)

SYMPTOM

“Is Ali above the class average?” → Ali 69.79, class also 69.79. The real class average was 74.75 (with that day's seed data).

ROOT CAUSE

In the parent account RLS shows only the child's grades; when the model computed a “class average” with plain SQL, it found the child's own average again. No error, just a confidently wrong answer.

FIX

Prompt: class/school-wide figures only through student_rank. Runner: scope_warning on aggregate results for parent/student. Eval: compare with the real value via truth_sql.

2 Conflicting rules: pass grade

Verified (eval)

SYMPTOM

“What is the pass grade?” → “It is not in our database, I cannot say for sure.”

ROOT CAUSE

The schema summary listed the pass grade under “DOES NOT EXIST in the database”, while the school policy was meant to be added. With two conflicting rules, the model refused.

FIX

The pass grade is school policy, not data: the PASS_GRADE setting (50). 50 was confirmed from the regulation; the 5-point conversion was not, so GRADE_LABELS stays empty.

3 The eval checker raised false alarms

Verified

SYMPTOM

The first full run showed 51/64; most were actually correct answers.

ROOT CAUSE

The number check only looked at the last query's rows and treated “74,10” and 74.1 as different numbers.

FIX

Every executed query is re-run under the same school_app and RLS context; numbers are compared by value, and row counts count as sources.

Bugs Found in Live Testing (2/3)

A partially fixed entry is here too, not hidden

		SYMPTOM	ROOT CAUSE	FIX
4	**Robotic tone, a retreating assistant** Partial	“I can't give advice” replies; answers with bold headings, bullets and a stock closing line.	The prompt was too cautious out of security concern; no conversational format was defined.	Short data-grounded support, a school-counsellor suggestion, opening with an understanding sentence, no stock closing. Open: the model can still speculate about causes.
5	**The model counts by itself** Verified (eval)	Says “4 students” from a 60-row list; the number is right but its source is not a query.	The tool result gave the model rows but no distinct counts; the model derived the number from the rows.	distinct_counts per column added to the tool result; the model now takes counts from row_count, total_students or distinct_counts.
6	**The eval ran blind** Verified	The eval ran for minutes without output; 429 responses were counted as failures.	The runner ignored Retry-After and printed no progress.	On 429 it waits and retries (at most 3 attempts per question); progress lines; answer texts and tool calls are written to the report.

Bugs Found in Live Testing (3/3)

Data and conversation quality

7 Inconsistent absence data

Verified

SYMPTOM

For Ali the recorded total was 3 against 10 non-attending records in attendance (old seed data); the assistant gave conflicting numbers.

ROOT CAUSE

The seed generated total_absences and the attendance records independently at random: 10 of 12 students mismatched.

FIX

total_absences is now the number of non-present attendance records; the schema summary defines the total and the detail; the status breakdown is delegated to SQL.

8 The seed changed the data on every run

Verified

SYMPTOM

After re-running the seed, Ali's average moved from 69.79 to 79.05 and the class average from 74.75 to 76.93.

ROOT CAUSE

The seed was not fixed and the data contained dates computed backwards from today; figures in the documents could silently go stale.

FIX

The data was frozen into database/mock_data.json with absolute dates; the seed loads from this file. The data digest (md5) is identical across three runs.

9 Parent-style address to a student

Verified (eval)

SYMPTOM

In a student account: answers like “talk to your child”.

ROOT CAUSE

The assistant prompt did not define the form of address per role.

FIX

Role-based address: parent “your child”, student “you”, teacher “your student”, principal neutral; a forbidden-phrase check in the eval.

Eval Result: 59 / 65

Full run; the fixed items were re-run afterwards

`d1`	Judge false alarm: the counts in the answer match the database exactly	**False alarm**
`c4`	Small model error: 73.98 truncated to “73” (74 when rounded)	**Known limitation**
`z2`	Relative dates (“yesterday/this week”): reference date added; passed on re-run	**Verified**
`m1`	One of two subjects was dropped: complete-answer rule; passed on re-run	**Verified**
`m5`	Open: causal speculation on an emotional topic	**Known limitation**
`s3`	The check was too strict: rule narrowed; passed on re-run	**Verified**

Process lesson: the server had been started without --reload before the fixes, so the eval measured old code; after a restart z2, m1 and s3 passed.

Changed and Developed Technologies

What we replaced this week, and why

Technology	What changed	Why
Tool-using assistant (tool use)	Instead of an SQL translator: the model calls the run_sql tool, sees the result, and writes the answer in natural language; the last 6 messages are context	Follow-up questions, honest answers when data is missing, a natural tone
Function allowlist (sqlglot)	An allowlist instead of a denylist; set_config, current_setting, pg_* are rejected	The privilege escalation through the model's SQL is closed
SECURITY DEFINER functions	student_rank and class_teacher: identity is read from the session, they cannot be called for another student	A parent sees their child's rank; nobody else's data is opened
Chat interface (Streamlit)	st.chat_message and st.chat_input; table and SQL in expandable sections	A conversation instead of a table
Eval set + LLM judge	65 questions; real values via truth_sql, tone and honesty via a judge	Assistant quality is measured repeatably
Frozen mock data	database/mock_data.json instead of a random seed; absolute dates	Figures and tests are reproducible
Policy through configuration	PASS_GRADE, REFERENCE_DATE, SHOW_SQL, CORS_ORIGINS settings	School policy lives in settings, not code
Login hardening	Uniform 401, per-username rate limit, allowed CORS origins	Brute force and username leakage

Week 7 — Scope and Acceptance Criteria

Delivered

- ✓ sql_safety.py — function allowlist
- ✓ Adversarial prompt tests
- ✓ Login: uniform 401, rate limit, CORS_ORIGINS
- ✓ eval/ — question set and runner
- ✓ docs/SECURITY.md — threat model

Acceptance criteria

1. SQL with set_config is rejected; a teacher never sees more than 4 students
2. A function not on the list is rejected, allowed ones run
3. An unknown user and a wrong password get the same 401
4. Login attempts are limited; CORS is configurable
5. The eval set runs and produces a report; tests pass without a key

Next Step — Week 8

Data Science I — Data and Exploratory Analysis

- Examining the UCI Student Performance dataset
- Exploratory data analysis (EDA)
- Feature engineering
- No model yet: only data preparation

Thank you